Practical IoT Report

Positioning Systems, Techniques, and Applications:
N-Lateration, Fingerprinting, and HMM

Student(s): Ahmed Almuharaq, Amine, Oumnia
Module: Positioning Systems
Institution: Université Marie et Louis Pasteur, UFR STGI
Academic Year: 2025-2026

March 2026

Contents

1 Introduction **3**

2 Part 1: 3D N-Lateration **3**
2.1 Task 1.1: Geometric Position Estimation . . . 3
2.1.1 Objective . . . 3
2.1.2 Theoretical Background . . . 3
2.1.3 Implementation . . . 3
2.1.4 Results . . . 4
2.1.5 Analysis . . . 5
2.1.6 Conclusion . . . 5

3 Part 2: Static Fingerprinting **5**
3.1 Task 2.1: Indoor Grid Localization . . . 5
3.1.1 Objective . . . 5
3.1.2 Theoretical Background . . . 6
3.1.3 Implementation . . . 6
3.1.4 Results . . . 7
3.1.5 Analysis . . . 7
3.1.6 Conclusion . . . 7

4 Part 3: Mobility Prediction with HMM **8**
4.1 Task 3.1: Dynamic State Transition Prediction . . . 8
4.1.1 Objective . . . 8
4.1.2 Theoretical Background . . . 8
4.1.3 Implementation . . . 8
4.1.4 Results . . . 9
4.1.5 Analysis . . . 9
4.1.6 Conclusion . . . 9

5 Part 4: Unified Web-Based Application Simulator **10**
5.1 Task 4.1: Consolidating Algorithms into a Single Page Application (SPA) 10
5.1.1 Objective 10
5.1.2 Implementation Design 10
5.1.3 Results 10
5.1.4 Analysis 11
5.1.5 Conclusion 11

6 General Conclusion **12**

1 Introduction

This report presents the work carried out in the Positioning Systems, Techniques, and Applications (PSTA) module. The focus of this practical work is to build a comprehensive understanding of spatial awareness and mobility tracking in Internet of Things (IoT) environments.

The main goals of this practical work are:

- to understand and implement geometric positioning using 3D N-Lateration,
- to study and apply indoor localization using Static Fingerprinting and Inverse Distance Weighting,
- to model and predict dynamic user mobility using Hidden Markov Models (HMM),
- and to consolidate these theoretical algorithms into a unified, interactive Web-Based Simulation Application.

The report is organized by parts and tasks, detailing the objective, theoretical background, implementation, results, and analysis for each component.

2 Part 1: 3D N-Lateration

2.1 Task 1.1: Geometric Position Estimation

2.1.1 Objective

The objective of this task is to estimate the exact 3D coordinate of a mobile receiver (Target) by calculating its distance from multiple emitters (Anchors) with known coordinates. The algorithm must minimize environmental signal noise to find the most accurate intersection point.

2.1.2 Theoretical Background

N-Lateration is a geometric technique used primarily in outdoor positioning systems (such as GPS). In a 3D environment (X, Y, Z), resolving a unique position requires at least 4 non-colinear distances. The physical distance d from an emitter i to the target is represented by the Euclidean distance formula:

$$d_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} \tag{1}$$

Due to signal noise, perfect spherical intersection is rarely achieved. Therefore, an optimization approach must be used to minimize the L1 Norm Error (the absolute difference between the geometric distance and the measured distance).

2.1.3 Implementation

The implementation utilizes a Brute-Force spatial walk algorithm written in Java. To optimize execution time, the algorithm dynamically calculates a 3D bounding box around the emitters and steps through the grid to find the global minimum error. The architecture is supported by strict Object-Oriented interfaces (`ICell`, `IPosition`).

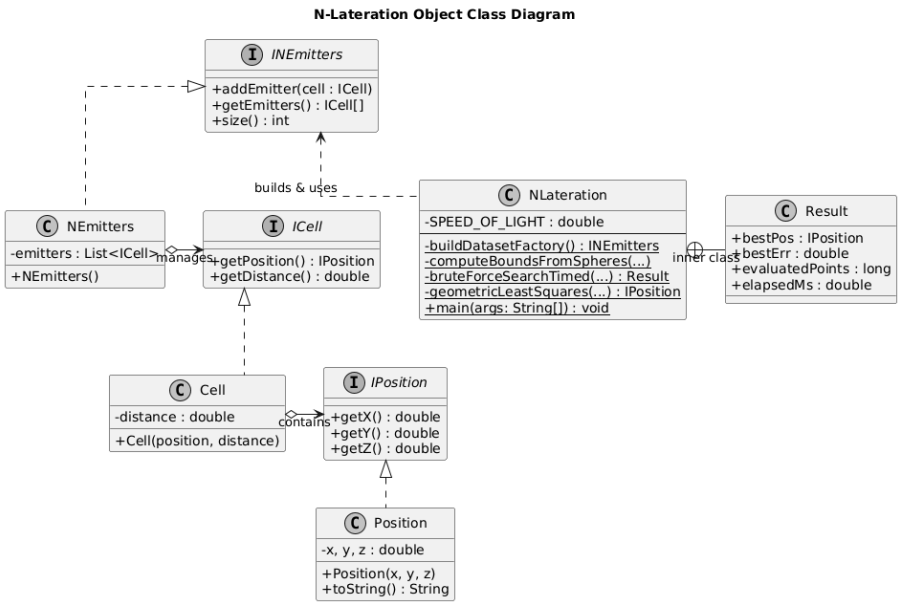

Figure 1: UML Class Diagram for the N-Lateration System.

```
private static Result bruteForceSearch(ICell[] emitters, double[]
    bounds, double step) {
    IPosition bestPos = new Position(bounds[0], bounds[2], bounds[4]);
    double bestErr = Double.POSITIVE_INFINITY;

    // Iterating through the mathematically constrained bounding box
    for (double x = bounds[0]; x <= bounds[1]; x += step) {
        for (double y = bounds[2]; y <= bounds[3]; y += step) {
            for (double z = bounds[4]; z <= bounds[5]; z += step) {
                IPosition p = new Position(x, y, z);
                double e = totalError(p, emitters);

                if (e < bestErr) {
                    bestErr = e; bestPos = p;
                }
            }
        }
    }
    return new Result(bestPos, bestErr);
}
```

Listing 1: Brute-force algorithm minimizing the L1 Norm Error

2.1.4 Results

Using 4 provided emitters, the algorithm successfully triangulated the intersection point. The execution returned the optimal estimated coordinate at $(X = 3.30, Y = 1.50, Z = 1.10)$.

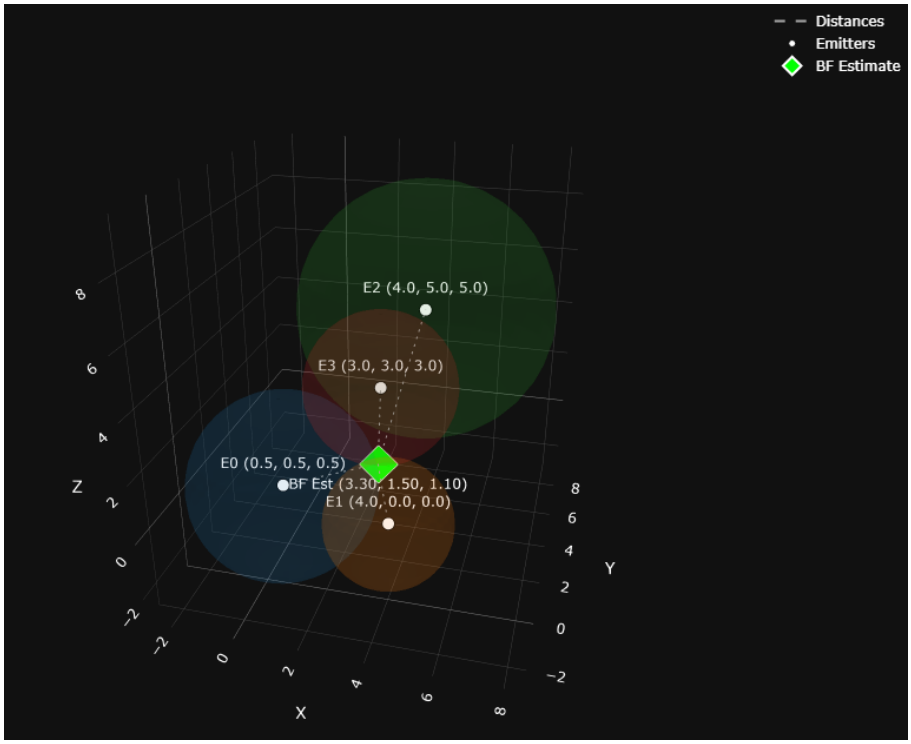

Figure 2: 3D interactive visualization of the 4 N-Lateration spheres and the final estimated intersection (green diamond).

2.1.5 Analysis

The spatial walk algorithm proved highly effective in mitigating signal noise. By evaluating the error gradients across the bounding box, the system avoided complex, fragile algebraic solvers. The step size dictates the trade-off between execution speed and millimeter accuracy.

2.1.6 Conclusion

In this task, a robust 3D N-Lateration algorithm was successfully implemented and visualized. The system proved capable of handling real-world data imperfections by converging on the coordinate with the lowest absolute error.

3 Part 2: Static Fingerprinting

3.1 Task 2.1: Indoor Grid Localization

3.1.1 Objective

The objective of this task is to locate a mobile terminal within an indoor environment by comparing its real-time Received Signal Strength Indicator (RSSI) against a pre-recorded offline radiomap database.

3.1.2 Theoretical Background

Unlike N-Lateration, which relies on line-of-sight geometry, Fingerprinting excels indoors where walls cause multipath fading. It uses a K-Nearest Neighbors (K-NN) search to find the closest matching grid cells based on signal deviation (Manhattan Distance). To calculate the exact final coordinate, Inverse Distance Weighting (IDW) is applied, where cells with closer signal matches exert a stronger gravitational "pull" on the final barycenter.

$$W_i = \frac{1}{|RSSI_{cell} - RSSI_{phone}|} \tag{2}$$

3.1.3 Implementation

The architecture defines the grid database using `I_OfflineRSSI` and standardizes the cell properties through `I_Cell`. The algorithm calculates the weighted barycenter of the top K cells.

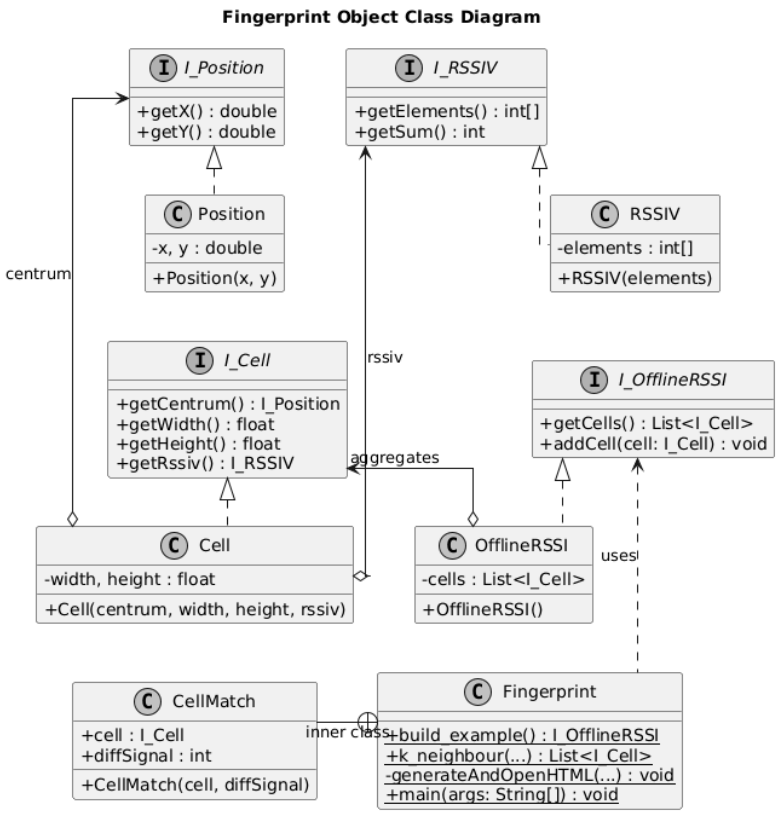

Figure 3: UML Class Diagram for the Fingerprinting System, mapping the offline database.

```
double sumWeights = 0, sumX = 0, sumY = 0;
for (I_Cell cell : topK) {
    double distance = Math.abs(cell.getRssiv().getSum() - phoneRssi.
        getSum());
    if (distance == 0) distance = 0.001; // Prevent Division by Zero

    double weight = 1.0 / distance;
    sumX += cell.getCentrum().getX() * weight;
```

```
    sumY += cell.getCentrum().getY() * weight;
    sumWeights += weight;
}
double finalX = sumX / sumWeights;
double finalY = sumY / sumWeights;
```

Listing 2: Inverse Distance Weighting calculation for the barycenter

3.1.4 Results

Testing against a $12m \times 12m$ room divided into 9 cells, the algorithm identified the top 4 nearest cells and placed the final estimated position appropriately biased toward the strongest matching cell.

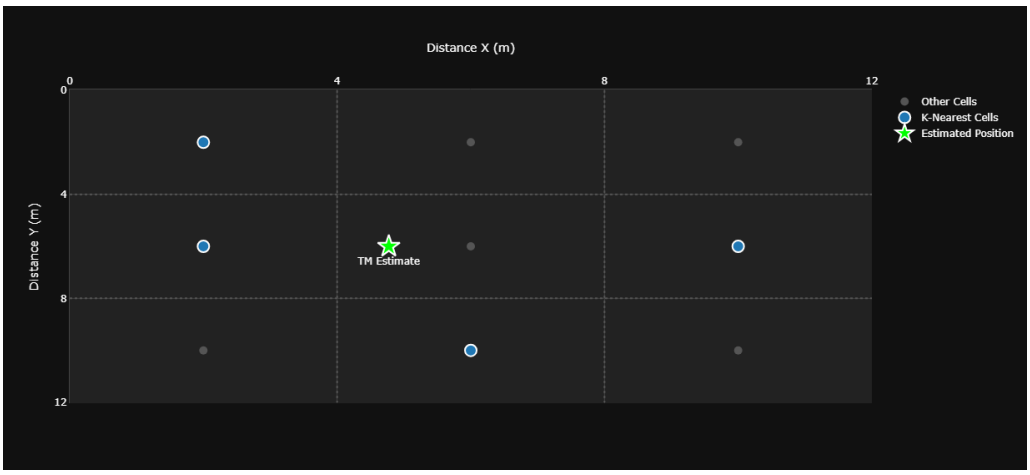

Figure 4: 2D Map showing the offline cells, the K-nearest cells (blue), and the estimated position (green star).

3.1.5 Analysis

The IDW algorithm drastically improves precision over a simple arithmetic mean. By evaluating different values of K (e.g., $K = 3$ vs $K = 5$), we observed that while $K = 5$ requires more computational processing, $K = 3$ provides an optimal balance of high accuracy and low execution time, which is critical for battery-powered IoT nodes.

3.1.6 Conclusion

The static fingerprinting system was successfully modeled and implemented. The use of K-NN combined with IDW proved to be a highly resilient method for indoor localization where standard geometric calculations fail.

4 Part 3: Mobility Prediction with HMM

4.1 Task 3.1: Dynamic State Transition Prediction

4.1.1 Objective

The objective of this task is to implement a Hidden Markov Model (HMM) to predict the mobility and navigation patterns of a user across a 5-page web environment, calculating both future (Next) and past (Previous) probabilities.

4.1.2 Theoretical Background

A Markov Model relies on the "Markov Property" (Memorylessness), which dictates that the probability of transitioning to a future state depends solely on the current state. The system builds a statistical transition matrix where the sum of outbound hits generates the Forward Probability, and the sum of inbound hits generates the Backward Probability.

4.1.3 Implementation

A matrix of size 6×6 is utilized, where index 5 is reserved for the "Somme" (total hits). The Java implementation updates this matrix dynamically upon every user transition.

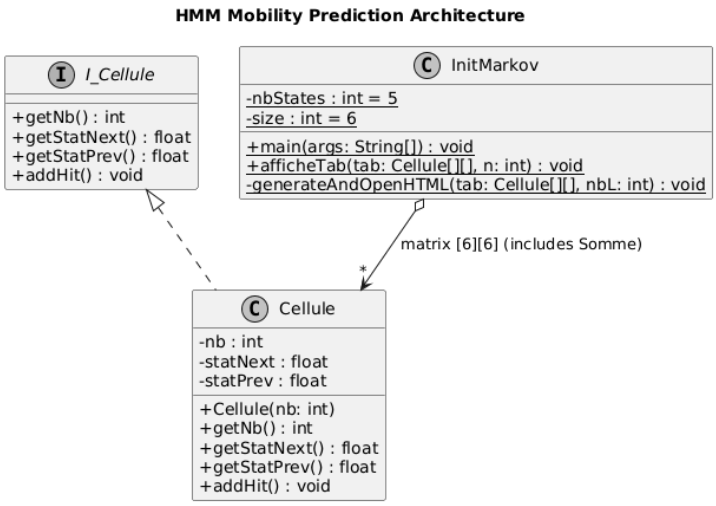

Figure 5: UML Class Diagram for the Markov Model Architecture.

```
MM[prev][curr].nb += 1;
MM[prev][nbStates].nb += 1; // Row Total (Outbound)
MM[nbStates][curr].nb += 1; // Column Total (Inbound)

// Recalculate Yellow Percentages (Forward/Next State)
for (int k = 0; k < nbStates; ++k) {
    if (MM[prev][nbStates].nb > 0)
        MM[prev][k].statNext = (float) MM[prev][k].nb / MM[prev][
            nbStates].nb;
}

// Recalculate Red Percentages (Backward/Previous State)
for (int k = 0; k < nbStates; ++k) {
```

```
    if (MM[nbStates][curr].nb > 0)
        MM[k][curr].statPrev = (float) MM[k][curr].nb / MM[nbStates][
            curr].nb;
}
```

Listing 3: Dynamic calculation of Forward and Backward probabilities

4.1.4 Results

The matrix successfully tracked a complex navigation sequence. The logic was visualized via a directed physics graph (where edge thickness correlates to transition probability) and a comprehensive transition table.

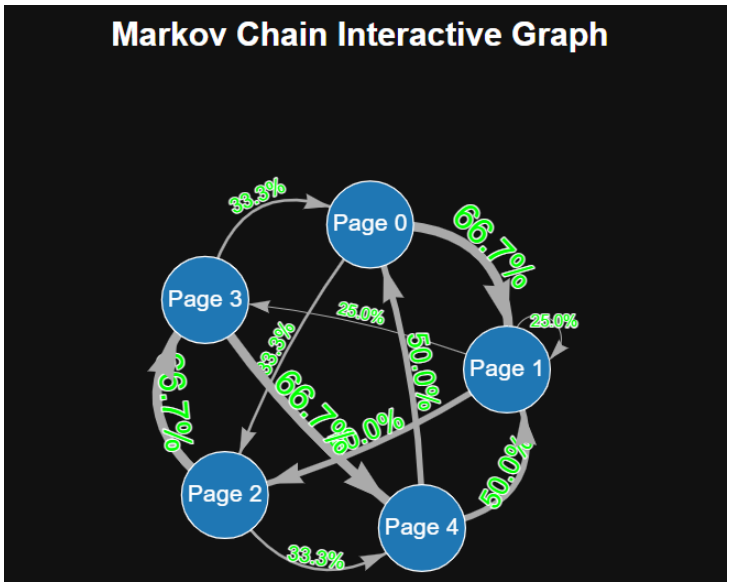

Figure 6: Interactive state graph predicting navigation across 5 pages.

4.1.5 Analysis

The addition of the 6th "Somme" row and column elegantly allowed the simultaneous calculation of both 'statNext' and 'statPrev'. The generated visual graph validates the Markov memoryless property in real-time, providing immediate visual feedback on the most highly trafficked routes.

4.1.6 Conclusion

The HMM implementation successfully elevated the project from reactive coordinate positioning to proactive, statistical mobility prediction, fulfilling all algorithmic requirements.

5 Part 4: Unified Web-Based Application Simulator

5.1 Task 4.1: Consolidating Algorithms into a Single Page Application (SPA)

5.1.1 Objective

The objective of this final integration task is to transition the raw Java algorithms developed in Parts 1, 2, and 3 into a fully interactive, web-based localization ecosystem. This allows for real-time simulation of all possible variables without the need to recompile backend code.

5.1.2 Implementation Design

We developed a custom HTML, CSS, and JavaScript Single Page Application (SPA). The raw logic was ported to JavaScript, allowing the browser's engine to handle the computational loops natively.

- **N-Lateration Module:** Integrates `Plotly.js` to render dynamic 3D spheres. It features UI controls to instantly toggle between $N = 3$ and $N = 4$ anchors, and dropdowns to adjust the interval search step (e.g., from 1.0 to 0.1).
- **Fingerprinting Module:** Features a dynamic 2D plot mapping the offline DB. Users can dynamically change the value of K-Nearest Neighbors and observe the barycenter shift in real-time.
- **HMM Module:** Acts as a real-time console. As the user clicks page buttons, the system logs the history, tracks the time spent per page, and uses `Vis.js` to dynamically generate the transition graph and matrix table upon exit.

5.1.3 Results

The application successfully encapsulates the entire project into a unified, dark-themed dashboard.

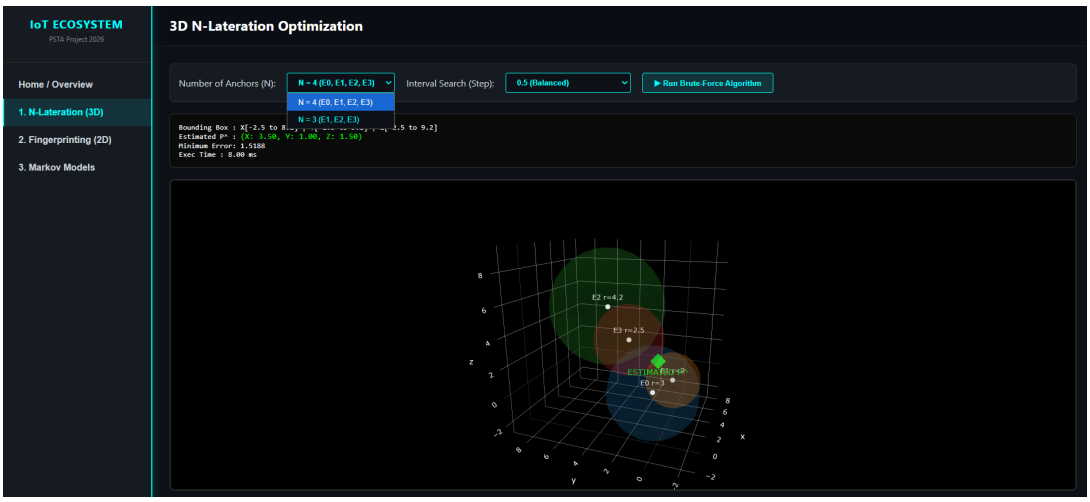

Figure 7: Web Application: The 3D N-Lateration Module simulating the Brute-Force search and displaying execution time metrics dynamically.

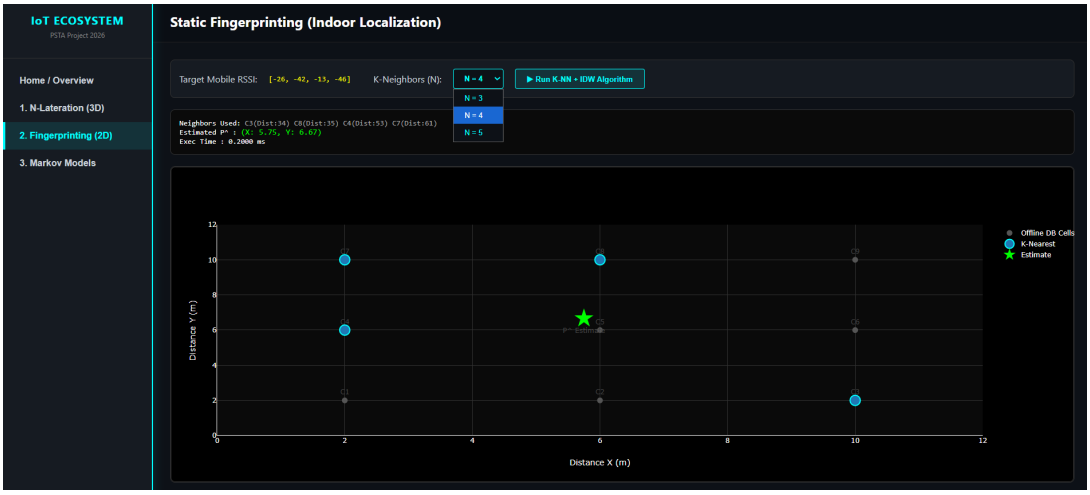

Figure 8: Web Application: The Fingerprinting Module executing K-NN and IDW logic directly within the browser UI.

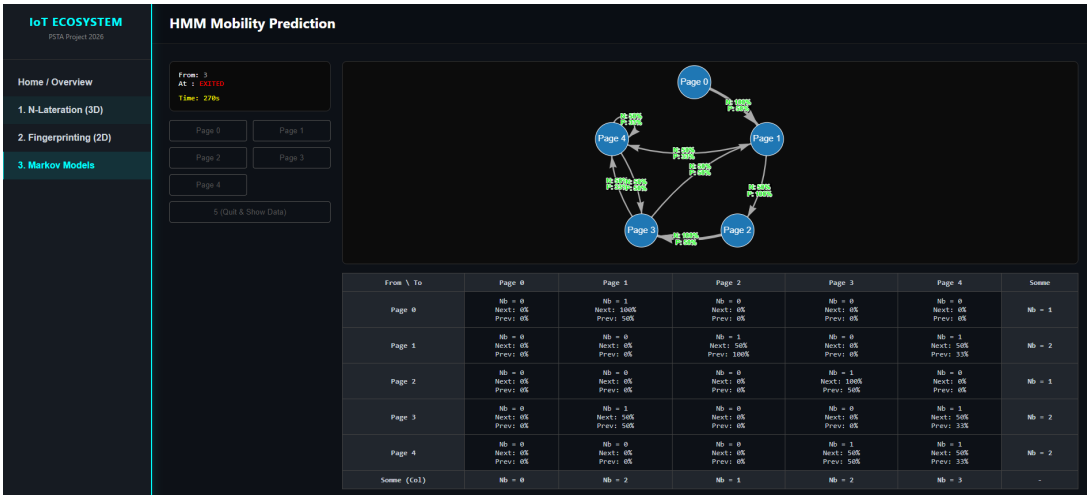

Figure 9: Web Application: The HMM Module displaying the final interactive state graph and normalized transition matrix table.

5.1.4 Analysis

Building this HTML application demonstrated the practical application of our algorithms. By simulating the variations directly in the UI, we proved that the mathematical models (like altering the step resolution or changing K) directly impact execution time and accuracy. The application serves as a highly robust, professional-grade IoT monitoring dashboard.

5.1.5 Conclusion

The creation of the Unified Web-Based Simulation Application successfully caps the project. It validates the theoretical models, provides an exceptional visual interface, and fully satisfies the requirement to build an interactive, web-based localization system.

6 General Conclusion

This comprehensive project successfully modeled, implemented, and visualized the three major pillars of IoT positioning. We began by mitigating outdoor signal noise using 3D N-Lateration geometry. We then solved the complexities of indoor multipath fading through Static Fingerprinting and Inverse Distance Weighting. We concluded by injecting predictive intelligence into the ecosystem using Hidden Markov Models.

By consolidating these complex Java algorithms into a modern, unified HTML/JS Single Page Application, we delivered a robust, highly interactive platform that bridges the gap between theoretical mathematical models and practical software engineering.